

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1
по курсу “Компьютерная графика”

Основы 3D графики

Выполнил: М. С. Мاستрюкова
Группа: М8О-304БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Познакомиться с основами 3D-графики: построением простых 3D-объектов, проекцией на 2D-плоскость, а также научиться работать с матрицами перспективы, ортогографической проекции и аффинными преобразованиями.

Вариант №2: правильный октаэдр

Дополнительные задания:

1. Обеспечить переключение матрицы проекции (ортогографическая и перспективная) в интерфейсе.
2. Добавить элементы интерфейса для изменения позиции, поворота и растяжения фигуры.
3. Добавить возможность движения и поворота фигуры по сложной траектории, сделать элемент интерфейса для паузы и воспроизведения «анимации», а также возможность изменять скорость «анимации» и другие параметры траектории.
4. Добавить возможность изменять цвет фигуры с помощью элементов интерфейса.
5. Назначить процедурно цвета для каждой вершины.

Метод решения

Работа выполнена на C++ с использованием Vulkan API совместно с библиотеками GLFW для создания окна и обработки ввода, ImGUI для построения графического интерфейса и GLM для линейной алгебры. Фигура задаётся шестью точками в локальной системе координат, расположенными на концах координатных осей: $+X$, $-X$, $+Y$, $-Y$, $+Z$, $-Z$. Каждая из восьми треугольных граней описывается тройкой индексов, ссылающихся на массив вершин. Всего в индексном буфере 24 значения. Сами данные загружаются в память GPU через буферы, которые создаются библиотекой VMA. Флаг аллокации позволяет VMA сразу предоставить указатель на память, доступную со стороны CPU.

Для передачи данных в шейдеры используются два uniform-буфера и два соответствующих набора дескрипторов: scene содержит матрицы view и proj, а model хранит модельную матрицу и цвет объекта.

Матрица вида строится через glm::lookAt, которая ставит камеру в точку (2.5, 2.0, 3.5) и направляет её в начало координат. Матрица проекции переключается в интерфейсе между двумя вариантами: glm::perspective даёт перспективу, glm::ortho — ортогографическую проекцию. В

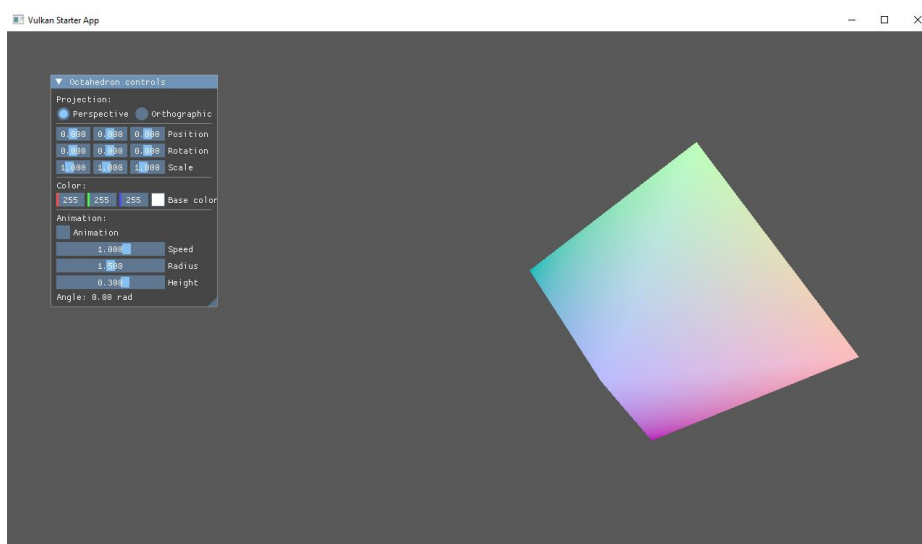
вершинном шейдере итоговая позиция вычисляется как произведение $\text{proj} * \text{view} * \text{model} * \text{vec4}(\text{position}, 1.0)$.

Каждой вершине при создании массива присваивается цвет, вычисленный из её координат по формуле $(\text{position} + 1) / 2$, так диапазон координат $[-1, 1]$ отображается в диапазон цвета $[0, 1]$, и разные вершины получают разные оттенки. Во фрагментном шейдере этот процедурный цвет умножается на цвет, выбранный пользователем в ColorEdit. В результате фигура сохраняет градиент между вершинами, а общий оттенок можно менять через интерфейс.

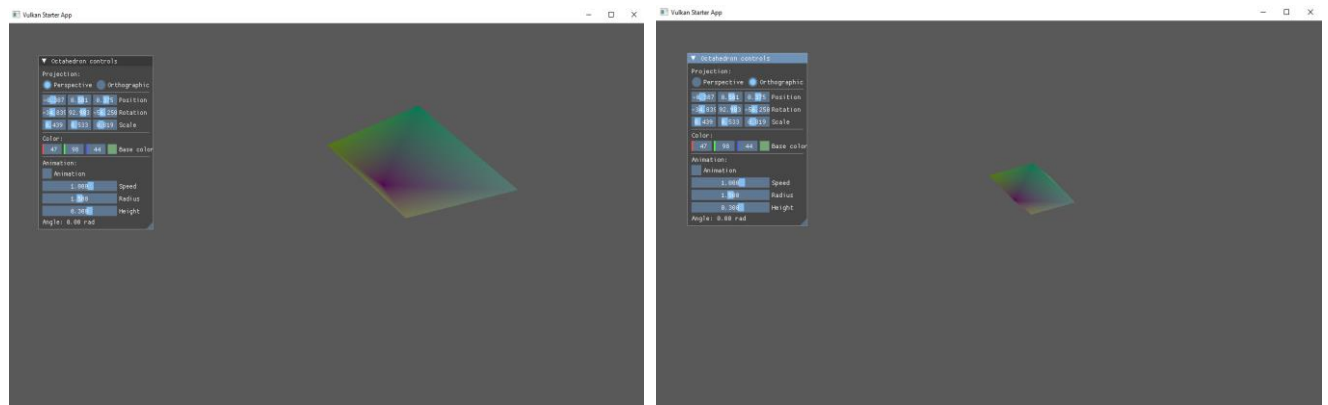
Использованные объекты Vulkan:

1. `VkBuffer`, `VmaAllocation`: вершинный, индексный буферы, а также Scene- и Model-uniform-буферы;
2. `VkDescriptorSetLayout`, `VkDescriptorPool`, `VkDescriptorSet`: привязка uniform-буферов к шейдерам;
3. `VkPipelineLayout` и `VkPipeline`: конфигурация графического конвейера: шейдеры, параметры растеризации, depth-тест;
4. `VkRenderPass`, `VkFramebuffer`, `VkCommandBuffer`: проход рендеринга и запись команд отрисовки;
5. `VkShaderModule`: загрузка скомпилированных .spv-шейдеров;
6. ImGUI: интерфейс для управления проекцией, трансформациями, цветом и анимацией;
7. GLFW: создание окна и обработка ввода.

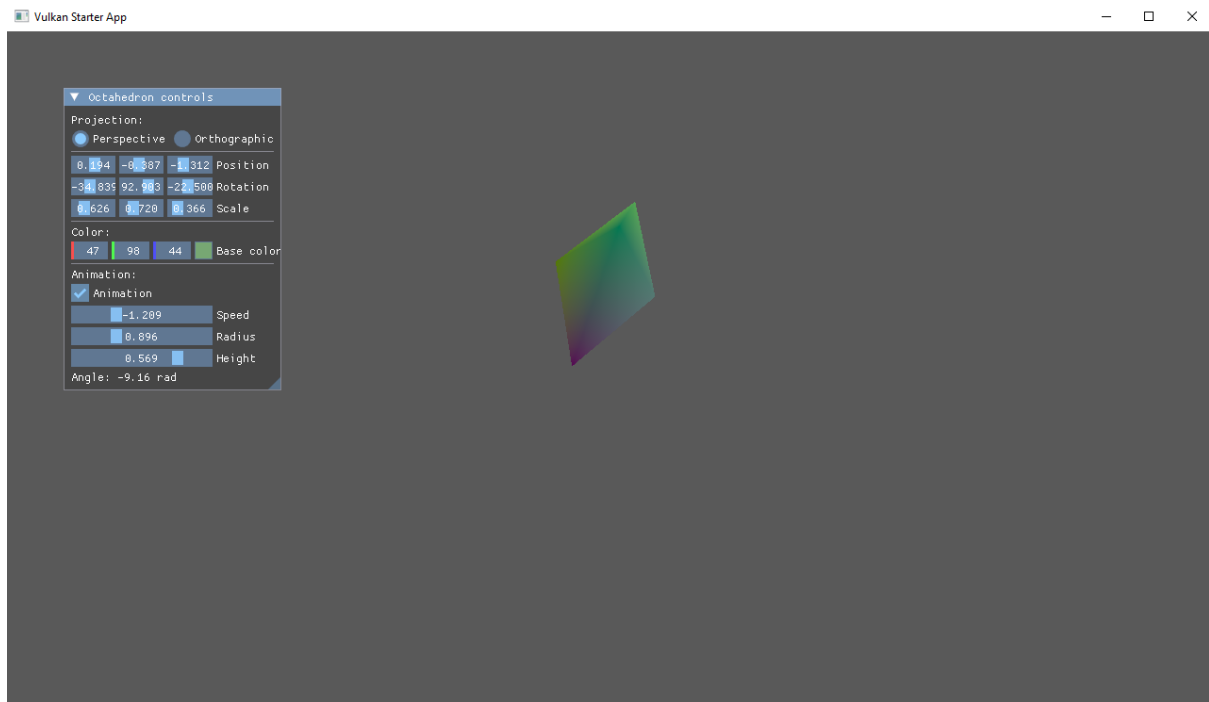
Результаты



1. Стартовое окно с фигурой и панель управления в ImGui с настройками проекции, трансформацией, цвета и анимации.



2. Изменение позиции, цвета и растяжения фигуры. Перспективная и ортографическая проекции.



3. Кадр во время анимации, изменены скорость, радиус и высота траектории.

Выводы

В ходе лабораторной работы я познакомилась с основами 3D-графики, научилась строить простые 3D-объекты, проекцию на 2D-плоскость, а также научилась работать с матрицами перспективы, ортографической проекции и аффинными преобразованиями. Освоила создание и настройку графического конвейера Vulkan: научилась загружать скомпилированные шейдеры, описывать входные атрибуты вершин, задавать параметры растеризации и связывать пайплайн с render pass и набором дескрипторов. Также я освоила математику 3D-преобразований, научилась собирать модельную, видовую и проекционную матрицы, понимать разницу между перспективной и ортографической проекциями и применять их в шейдере, и научилась строить интерактивный интерфейс на ImGui

Программа работает в реальном времени, поддерживает динамическую смену проекции, ручные трансформации, анимацию по сложной траектории с настройкой её параметров, изменение цвета и процедурные цвета вершин.